TornWatch

A Durable, Temporal-Powered Monitoring & War-Support Toolkit for Torn City

Project Specification

Version 0.2 — Draft for review

Table of Contents

How to view this Table of Contents

This is a real, auto-generated Table of Contents field, not static text — it will look empty until you refresh it once. In Google Docs: open the document, click into the TOC, and click the refresh icon that appears (or Insert > Table of contents > update). In Microsoft Word: right-click the TOC and choose "Update Field," then "Update entire table." After that one refresh, it will populate correctly from this document's headings and stay accurate as the doc changes.

1. Overview

TornWatch is a monitoring and faction war-support tool for the browser game Torn City, built on Temporal, a durable execution platform. It is read-only and informational by design: it watches game state and notifies players, rather than automating in-game actions. This keeps it compliant with Torn's API Terms of Service and avoids any risk to player accounts.

1.1 One-sentence pitch

A background service and Discord bot that watches a Torn City character and faction — personal cooldown timers, faction war chains, hit-quality during wars, and item market listings — and reliably notifies players, using Temporal to guarantee it keeps working correctly through crashes, network outages, and Torn's API rate limits.

1.2 Why Temporal (and not just a cron job)

Several of this project's core features have a shape that plain scripts handle poorly but that Temporal is purpose-built for:

- Processes that must stay alive and correct for hours (a faction chain) or weeks (a standing monitor), surviving laptop sleep, process crashes, and deploys without losing state.
- Built-in retry and backoff for a third-party API with a hard, documented rate limit (Torn allows up to 100 requests per minute per user) and well-defined transient error codes.
- Race conditions between a deadline (a chain timeout) and external events (a player reporting a hit via Discord), which map directly onto Temporal's Timers and Signals.
- Fan-out across many faction members (Child Workflows) with independent state and independent failure handling.
- A live, self-managed queue of participants who can join or leave mid-event, which needs durable, race-safe state — not just a flat timer.

2. Goals & Non-Goals

2.1 Goals

- Build a genuinely useful personal and faction tool for an active Torn City player and their faction.
- Support real faction war operations: chain timing, hit rotation, and target quality, with Discord as the primary interface for live coordination.
- Learn Temporal's core primitives hands-on: Workflows, Activities, Workers, Timers, Signals, Schedules, and Child Workflows.
- Stay strictly within Torn's API Terms of Service throughout.

2.2 Non-Goals (at least for v1)

- No automated gameplay actions (auto-attacking, auto-buying, auto-training). The Torn API is read-only by design — there is no endpoint to place an order or commit an attack — and Torn's stated goal for the API is informational, explicitly not to give players an advantage over others.
- No scraping the game's web pages (HTML scraping) — official API only.
- No multi-tenant hosted product for other factions in v1 — this is a personal/faction tool first.
- No mobile native app — a PWA (installable web app) is the long-term front-end target instead, alongside the Discord bot.

Important constraint — Terms of Service

Torn's API documentation states the system is meant to provide read-only, informational access, and that Torn does not want the API used to create an unfair gameplay advantage. Torn also expects any tool that stores a player's API key to clearly disclose what is stored, who can see it, and why. Every feature in this spec is designed to read data and notify players — never to submit actions on a player's behalf.

3. Temporal Concepts Used in This Project

A quick-reference glossary for the Temporal vocabulary this spec relies on, with a note on which feature first introduces each concept.

Concept	Plain-language definition	First used in
Workflow	Durable orchestration code describing what should happen, step by step. Survives crashes by replaying recorded history instead of losing progress.	Phase 0
Activity	The only place allowed to do real-world work: HTTP calls, reading the clock, anything non-deterministic. Workflows call Activities, never the network directly.	Phase 0
Worker	A long-running process that polls Temporal for work and actually executes Workflow/Activity code.	Phase 0
Retry Policy	Built-in configuration for how an Activity should retry on failure (backoff timing, max attempts) — replaces hand-written retry loops.	Phase 1
Timer (durable sleep)	A Workflow can sleep for minutes or hours and wake up exactly on schedule, without polling, and without losing the sleep if the process restarts.	Phase 1
Signal	An asynchronous message sent into a running Workflow from the outside world (e.g. a Discord command reporting a hit), without restarting the Workflow.	Phase 2

Query	A way to read a running Workflow's current in-memory state from the outside, without affecting it (e.g. "who's next in the queue right now?").	Phase 2
Schedule	Tells Temporal to start a Workflow automatically on a recurring basis (e.g. every 5 minutes), similar to cron but durable and observable.	Phase 3
Child Workflow	A Workflow started by another Workflow, with its own independent history and failure handling — used for fan-out (one per faction member).	Phase 3
Saga / Compensation	A pattern for multi-step processes where, if a later step fails, earlier steps are explicitly undone or accounted for.	Phase 4

4. Architecture Overview

At a high level, TornWatch consists of these moving pieces:

Component	Responsibility
Temporal Service	Either the local Temporal CLI dev server (development) or Temporal Cloud (later, if desired). Stores Workflow Event History and coordinates Workers.
Worker process	A long-running Node.js (TypeScript) process that polls the Temporal Service and executes TornWatch's Workflow and Activity code. Runs continuously in production.
Activities	All Torn API calls, notification sends, and any other I/O. Organized by domain: torn-api.ts, notifications.ts, discord.ts.
Workflows	Orchestration logic: per-character monitor, chain watcher, faction hit-quality monitor, market watcher. No direct I/O — calls Activities only.
Discord bot	A separate long-running process connected to Discord's gateway. Posts notifications driven by Workflow events, and turns Discord commands/button-clicks into Signals sent to running Workflows via the Temporal Client.
Client / Front-end	Starts Workflows, sends Signals, runs Queries. Starts as a CLI/small script, later becomes a PWA web app and/or the Discord bot itself.

4.1 Why a single Worker is enough

Unlike a typical web server that must scale horizontally under load, TornWatch's Worker only needs to keep up with a handful of Workflows (one per tracked character, one per active chain, one per faction). A single Worker process is sufficient for the entire project's scope, including the faction fan-out phase.

5. Feature Specifications

5.1 Personal Cooldown & Timer Notifications

Tracks a player's personal time-sensitive stats and notifies them (via Discord) when something needs attention, without requiring them to keep the game open or check manually.

Tracked signals

- Energy and nerve bars reaching full (or a configurable threshold).
- Travel timer landing.
- Drug cooldown expiring (and other addiction-related cooldowns Torn tracks).
- Hospital and jail release timers.
- Any other periodic bar/cooldown the Torn API exposes that the player wants to track — designed as a configurable list, not a hardcoded set, so new stats can be added without code changes.

Behavior

One Workflow Execution per tracked character. Rather than polling constantly, the Workflow computes the next relevant wake time across all tracked stats and sleeps using a durable Timer until then, waking early only if a Signal (e.g. a config change) arrives first.

5.2 Faction Chain Watcher

Supports a faction war chain — a multi-hour event with a countdown timer that resets on every hit and ends the chain if nobody attacks in time. This is the most state-rich feature in the project: it manages a live, self-service participant queue, a parallel "Chain Leader" track, and a shared countdown timer, all coordinated through Discord.

5.2.1 The hit rotation (queue)

- Players join or leave the rotation at any time via Discord bot commands (e.g. /join-chain, /leave-chain) — this is not a fixed pre-set schedule.
- Order is first-come-first-served: position is determined by when a player joined relative to others currently in the queue.
- The bot proactively notifies the next player or two in line before their turn arrives, so they have time to be ready.
- When a player hits, the Chain Watcher Workflow receives a Signal, advances the queue, and resets the chain countdown Timer.

5.2.2 The Chain Leader track

Higher-level players often need to operate outside the normal rotation — hospitalizing the strongest currently-online members of the enemy faction so they can't retaliate or counter-chain, rather than taking a turn in order.

- Becoming a Chain Leader is self-declared via a Discord command (e.g. /become-chain-leader) — there is no fixed pre-assigned list.
- Chain Leaders hunt independently, generally targeting tough enemy online members rather than following the queue.
- In v1, a Chain Leader remains part of the regular rotation at the same time ("both" model) — this matters when the regular queue is small and needs every available hitter. Fully separating the two roles (removing a Chain Leader from the queue while active) is a planned future option once queue depth makes that safe.
- Hits from a Chain Leader still reset the shared chain countdown Timer, exactly like a regular rotation hit.

5.2.3 Timer-reset behavior on out-of-rotation hits

A key open design question was whether a hit landed outside the normal rotation order (e.g. by a Chain Leader) should also skip or reorder the regular queue. The v1 default, chosen deliberately to avoid unfairness around in-game pay-per-hit incentives, is:

- Out-of-rotation hits reset the chain countdown Timer only. The regular queue's order and "whose turn" pointer are unaffected — nobody in the queue is skipped because someone else hit.

> **Planned future enhancement — dynamic / configurable reset behavior**
> The right reset behavior may depend on factors like how many enemy targets are currently available (not hospitalized, not traveling) versus how many players are waiting in the queue. A future version should make this behavior dynamic or at least configurable — for example, a toggle accessible through a Discord command or a future web dashboard — rather than a single hardcoded rule. This is intentionally deferred past v1.

5.2.4 Temporal design notes

- The queue, the Chain Leader set, and the countdown deadline are all state held inside a single Chain Watcher Workflow Execution per active chain — not a separate database — since this state only needs to exist while the chain is active.
- Joining, leaving, hitting, and declaring Chain Leader status are all modeled as Signals into the running Workflow.
- A Query exposes current state (time remaining, current queue order, who's flagged as Chain Leader) so the Discord bot and any future dashboard can read live status without disturbing the Workflow.

5.3 Market Watcher

Watches a player-configured list of specific items across the Torn item market and bazaars, and notifies the player when one is listed meaningfully under typical value — framed strictly as faster signal detection for a human decision, since the Torn API has no order-placement endpoint and Torn does not want tools that create an unfair advantage.

Behavior

- Player selects specific item IDs to watch and a target/threshold price (or a percentage-under-typical-value rule).
- An Activity polls market and bazaar listings for watched items on an interval respecting Torn's rate limit and response caching window.
- When a listing appears under the configured threshold, a short multi-step sequence detects the listing, re-confirms it's still present a few seconds later (listings can be stale or already gone by the time they're seen), and then sends a notification — a natural fit for a Saga-style compensation step if the notification fails after the listing was already confirmed.

5.4 War / Hit-Quality Tracker

Checks whether faction members (on both sides of a war) are attacking targets that make sense, rather than only farming much weaker opponents.

> **Design note — respect is per-matchup, not a fixed stat**
> In Torn, the respect value of an attack is calculated from the relationship between attacker and defender for that specific matchup — it isn't a fixed property of a player that can be looked up directly. This feature therefore evaluates each outgoing attack's matchup (attacker vs. defender stats/level at the time of the hit) rather than treating "respect" as a static field on a player record.

Behavior

- For each recent attack by a tracked faction member, evaluate the matchup against the defender to estimate whether it was a high-value or low-value target choice.
- Flag players who are repeatedly choosing low-value matchups (much weaker opponents) instead of viable higher-value targets that were available.
- During an active war, this can run on a Temporal Schedule across the whole faction using one Child Workflow per member, fanning out efficiently.

---

6. Discord Bot Integration

The Discord bot is the primary live interface for the Chain Watcher and the main notification channel for personal timers and market alerts. It is a separate long-running process from the Temporal Worker, connected to Discord's gateway, that talks to the Temporal Client to send Signals into running Workflows and to read Queries for live status.

6.1 Responsibilities

- Personal notifications: DM or mention a player when one of their tracked timers (energy, nerve, drug cooldown, travel, etc.) is ready.

- Chain queue management: /join-chain, /leave-chain, /become-chain-leader, and a command or reaction to report a hit, each translated into a Signal sent to the active Chain Watcher Workflow.
- Turn notifications: proactively message the next player (or two) in the rotation before their turn arrives, using the Workflow's Query results.
- Target suggestions: when relevant, surface a suggested target for the player whose turn is next, informed by the hit-quality logic in 5.4.
- Market and war alerts: post notifications from the Market Watcher and War Tracker into a faction channel or as DMs, depending on configuration.

6.2 Open design questions

#	Question	Current thinking
1	Bot framework choice (discord.js vs. Discord.py vs. other)	Leaning discord.js to stay in TypeScript end-to-end with the rest of the project, avoiding a language switch.
2	Where does the bot process run relative to the Worker?	Both as long-running processes on the same home desktop in production; can run on the development machine locally for testing.
3	How does the bot authenticate Discord users to Torn identities?	Needs a one-time link step (e.g. a /link-torn-id command tying a Discord user to a Torn player ID) — to be designed in detail during this phase.
4	Should hit-reporting be a slash command, a button under the notification message, or both?	Likely both — a button is faster for the common case; a command works if the original message has scrolled away.

7. Phased Roadmap

Each phase below is independently demoable — it should be possible to stop after any phase and still have something real and working.

Phase 0 — Foundations

Prove the Worker, Temporal Service, and Client wiring works end-to-end against a real Torn API call, before any game logic exists.

- Local Temporal dev server running via the Temporal CLI.
- One Activity calling Torn's /user endpoint; one Workflow calling that Activity once.
- Done when: a Client script prints real player data fetched through a Workflow Execution, visible in the Temporal Web UI's Event History.

Phase 1 — Personal Timer Notifications

Build the full feature described in section 5.1: per-character Workflow tracking energy, nerve, travel, drug cooldown, and similar timers, sleeping via durable Timers and notifying through a simple channel (Discord webhook is the simplest starting point before the full bot exists).

- A real Retry Policy tuned to Torn's documented error codes — rate-limit and transient errors retry with backoff; invalid-key errors do not retry endlessly.

> **Suggested chaos test**
> Kill the Worker process while a Workflow is mid-sleep, then restart it. Confirm the Workflow resumes and still fires the notification at the correct original time.

Phase 2 — Chain Watcher (core logic, polling-based)

Build the Chain Watcher Workflow described in section 5.2 — the queue, the Chain Leader track, and the shared countdown Timer — using simple polling of Torn's faction attack/news log to detect hits, before the Discord bot exists. This proves the state machine and Signal/Query logic in isolation.

Phase 2.5 — Discord Bot Integration

Build the bot described in section 6: queue join/leave/hit-reporting commands, Chain Leader self-declaration, turn notifications, and personal timer notifications migrating from webhook to bot. This replaces Phase 2's polling-based hit detection with real-time Signals from Discord, and resolves the Discord-to-Torn-identity linking question.

Phase 3 — Faction-Wide Hit-Quality Monitor

Build the War Tracker from section 5.4: a Parent Workflow for the faction with one Child Workflow per member, run on a Temporal Schedule, using batch-efficient API calls where possible to conserve the shared rate-limit budget.

Phase 4 — Market Watcher

Build the feature from section 5.3, including the detect-confirm-notify sequence and its Saga-style compensation path.

Phase 5 — PWA Front-End

A web dashboard (reusing the existing TypeScript codebase) showing live chain state, tracked timers, and recent alerts, installable as a Progressive Web App, alongside the Discord bot rather than replacing it.

8. Tech Stack

Layer	Choice	Notes
Language	TypeScript (Node.js)	Used for Workflows/Activities, the Discord bot, and the eventual PWA front-end, avoiding a language switch.
Orchestration	Temporal (self-hosted dev server, or Temporal Cloud)	Local temporal server start-dev is sufficient for the entire project.
External API	Torn API v2 (official, read-only)	All game data access. No HTML scraping.
Discord	discord.js	Long-running bot process, separate from the Worker, talking to the Temporal Client.
Persistence	None required for core logic	Temporal's Event History is the source of truth for Workflow state. A lightweight DB could be added later purely for the dashboard's historical reporting.
Front-end (Phase 5)	PWA (vanilla or lightweight framework)	Installable web app; talks to the Temporal Client via a small backend API layer.
Hosting (production)	Home desktop	Worker and Discord bot processes both run continuously.

9. Suggested Project Structure

```
tornwatch/  src/    activities/      torn-api.ts         (all Torn API calls)
notifications.ts    (Discord/email sends)    workflows/
character-monitor.ts      chain-watcher.ts      faction-hit-monitor.ts
market-watcher.ts    bot/      index.ts            (Discord bot entrypoint)
commands/           (slash commands)    worker.ts    client.ts  package.json
tsconfig.json  .env (gitignored — API keys, bot token, webhook URLs)
```

10. Torn API Reference Notes

Fact	Detail

Rate limit	Up to 100 individual requests per minute, per user, across all of that user's API keys combined.
Response caching	Each distinct call is cached for 29 seconds; calling more often than every ~30 seconds for the same selection returns stale cached data.
Key access levels	Four levels: public, minimal, limited, full. A custom key scoped to only the needed selections is recommended to minimize exposure if the key is ever compromised.
Relevant error codes	2 = incorrect key, 5 = too many requests, 8 = temporary IP block, 13 = key disabled due to owner inactivity, 17 = backend error. Codes 5, 8, and 17 are good candidates for automatic retry; code 2 should alert the user rather than retry.
Faction-wide efficiency	Prefer one faction-wide endpoint call over iterating individual member calls where possible, to conserve the shared request budget during fan-out.
ToS posture	API is explicitly intended to be read-only/informational. Storing another player's key requires clear disclosure of what's stored and why.

11. Risks & Open Questions

#	Question / Risk	Current thinking
1	Dynamic/toggleable chain-timer reset behavior (section 5.2.3)	Deferred past v1; revisit once queue depth and target-availability data exist to inform the rule.
2	Mutually-exclusive Chain Leader vs. regular queue membership	v1 allows both simultaneously; revisit once queue depth makes full separation safe.
3	Discord-to-Torn identity linking	Needs a one-time link command; security/verification approach to be designed in Phase 2.5.
4	API key and bot token storage	Local .env file, gitignored, never committed or logged in plaintext; GitHub Actions secrets if CI is added later.
5	Is a database needed?	Not for Phases 0–4 — Temporal's Event History covers Workflow state. Revisit only if the dashboard needs historical reporting beyond what a Query can expose live.

12. Immediate Next Steps

1. Review this spec and adjust phase order/scope as needed — nothing here is final.
2. Confirm local dev environment setup (Node.js, Temporal CLI) on the machine that will run development work.

3. Generate a scoped Torn API key (custom access, limited to needed selections) rather than a full-access key.
4. Build Phase 0 and confirm end-to-end wiring against a real Torn API call.
5. Proceed phase by phase, migrating the long-running Worker and bot processes to production hosting once Phase 1 needs multi-hour uptime.

— End of specification —